

Mikhail Razakov

C++ / Program Analysis Engineer

I build analyzers and reusable systems components in C++ and C. The evidence includes the PS-form Analyzer with an exact oracle, 12 AdvancedAlgorithms modules, hand-written NASM and a practical IA-32 course, plus an x86-64 codegen/register-allocation laboratory.

- Moscow / remote
- Available from September 2026
- Up to 20 hours per week
- HSE University student

misha13022008@gmail.com · LmaMicody · github.com/Misha1302 · LinkedIn

1st out of 49 the only 104/104 result in the MCST selection	C++23 · strict builds warnings-as-errors, ASan/UBSan, clang-tidy	Exact oracle + metamorphic tests WATL hunting and counterexample retention
---	--	--

2026 MCST · LLVM / C++23	C17 analyzer normalization + exact/conservative strategies	C++23 graph core RPO, shortest paths, SCC, max-flow	x86 / x86-64 Assembly NASM/FASM · ABI · x87 · codegen
------------------------------------	--	---	---

01 · EXPERIENCE

C++ and systems projects.

JULY–AUGUST 2026	MCST — Compiler Engineering Intern LLVM track · C++23 · remote - Built a shared directed-graph core and four components: RPO with cycle detection, Dijkstra, Tarjan SCC, and Dinic max-flow. - Defined graph-input and unknown-node error contracts; handled self-loops, parallel edges, unreachable vertices, and disconnected components. - Configured C++23/CMake builds with warnings-as-errors, ASan/UBSan, clang-tidy, and automated IO tests.	
2024 → PRESENT	UniversalToolchain / Wist2 — Creator and Primary Developer Compiler/runtime architecture · verification engineering - Designed multiple IR layers, two execution paths, and an opt-in AIR → SSA → AIR route. - Added capability contracts, structural verifiers, and interpreter/CIL parity tests. - Verified baseline dated July 14, 2026: 75 .NET projects, 1,325 tests, 0 warnings and 0 errors.	
2021 → PRESENT	Programming Tutor C/C++ · Python · NASM · algorithms	Taught about 50 students in C/C++, Python, NASM, and algorithms; I review code and explain execution models.

02 · SELECTED WORK

Program analysis, C++ components and low-level backend experiments.

C17 · PYTHON · PROGRAM ANALYSIS PS-form Memory Dependence Analyzer A conservative cross-iteration memory-dependence analyzer: yes and no require proof; unresolved cases remain maybe. GitHub ↗	Result: 1st out of 49, the only 5.0/5.0 and 104/104 result. Algorithms: normalization, range/residue/GCD filters, exact affine analysis, monotonic scan, and cost-bounded enumeration. Testing: exact oracle for small domains, randomized and metamorphic tests, and WATL hunting. Safety: a heuristic may confirm a conflict, but failure to find one cannot produce a false no.
--	---

C++23 · HEADER-ONLY LIBRARY AdvancedAlgorithms Reusable algorithms and data structures with explicit contracts, reference oracles and large-input tests. - Centroid decomposition, HLD, LCA, Dinic, iterative Tarjan, bridges/articulation points and Dijkstra. - DSU, lazy segment tree, sparse table, Aho-Corasick and simulated annealing. - ASan/UBSan, differential testing and checks up to 500k vertices. GitHub ↗	ASSEMBLY AND BACKEND · NASM · RUST · IA-32 / X86-64 x86 Assembly, Codegen & Register Allocation Hand-written NASM for IA-32 and an x86-64 code-generation lab, spanning ABI and stack mechanics through liveness, register allocation and native-code validation. - NASM IA-32: cdecl, stack frames, libC calls, structures and addressing, x87; buildable examples and a practical course. - x86-64: SSA-like IR, live intervals, interference graph, linear scan and lowering/emission through iced-x86. - Disassembly and compiler-output analysis: differential tests, spill/code-size metrics and isolated execution. Codegen lab ↗ NASM course ↗ NASM/FASMIA-32 / x86-64ABI / x87	C# / .NET · IR/SSA · RUNTIME UniversalToolchain / Wist2 A public compiler/runtime framework with multiple IRs and two execution backends. - AST, Bytecode, AIR, interpreter and CIL. - AIR-to-SSA lowering/emission, constant folding, SCCP-lite, branch/unreachable-block folding and DCE. - Capability contracts, structural verifiers and parity tests. GitHub ↗ Technical paper · RU PDF
--	--	---

03 · SKILLS

C++ systems, program analysis, and compiler infrastructure.

Core	C++23, C17, Python, Linux, CMake, Git
Algorithms	Centroid/HLD/LCA, Dijkstra, SCC, bridges/articulation points, Dinic, DSU, segment tree, sparse table, Aho-Corasick, symbolic and affine analysis
Testing	Conservative semantics, exact oracles, randomized/metamorphic testing, sanitizer builds, counterexample retention
Low-level / Assembly	NASM/FASM, IA-32 and x86-64, cdecl and System V AMD64 ABI, stack frames, libC interop, structures/addressing, x87, GDB/objdump, compiler-output analysis
Additional	C#/ .NET compiler/runtime architecture, GitHub Actions, structured diagnostics

04 · RECOGNITION

Olympiad and project achievements.

2025–2026	Programming olympiads Prize-winner of the HSE “Vysshaya Proba” Olympiad in competitive and industrial programming; prize-winner of the regional Russian Informatics Olympiad.	Vyshaya Proba ↗
2025–2026	Compiler projects Two highest-tier diplomas in the MEPHI Junior competition and a highest-tier diploma in the Baltic Science and Engineering Competition.	MEPHi Junior 2025 ↗ MEPHi Junior 2026 ↗ Baltic competition ↗
2025–2026	HSE “Vysshiy Pilotazh” · Computer Science Winner of the Computer Science track.	Results ↗

05 · EDUCATION

Education and availability.

STUDENT	HSE University Moscow	HSE University student.
FROM SEPTEMBER 2026	Part-time availability Moscow / remote · up to 20 hours per week	Looking for C++ systems, program analysis, compiler infrastructure, or verification-tooling work.